# Unsupervised Visual Anomaly Detection for Industrial Quality Inspection

## Accuracy–Latency Comparison of PatchCore and EfficientAD

**Mini Project Presentation**


**SUBMITTED BY:**

1. Pratik Sawant (PRN: 20240802324)
2. Anvesh Patil (PRN: 20240802334)
3. Jayesh Deshmukh (PRN: 20240802335)

**ACADEMIC & GUIDE DETAILS:**

Class / Division: TY B.Tech (AI & ML)

Department: School of Computer Science, Engineering & Applications

Guided by: [Faculty In-Charge Name]

Institution: D Y Patil International University, Pune

Academic Year: 2026–2027

# Introduction



## Project Background & Purpose

• Industrial manufacturing requires strict zero-defect quality control across fast-moving production lines.

• Visual inspection is the primary barrier preventing flawed parts from reaching customers or critical assemblies.

• Our project introduces an automated computer vision inspection framework that detects surface anomalies without needing defective training data.

• We model the normal visual manifold using one-class unsupervised learning to identify unseen flaws.

## Comparative Research Focus

• Conducts an empirical accuracy-latency trade-off comparison between two state-of-the-art vision paradigms:

• 1. PatchCore (Roth et al., CVPR 2022): Memory-bank non-parametric modeling with 0 training epochs.

• 2. EfficientAD (Batzner et al., WACV 2024): Lightweight student-teacher distillation with sub-10ms latency.

• Delivers an interactive Streamlit dashboard with JET heatmaps and SQLite audit logging.

# Problem Statement



## The Core Problem & Current Practices

• What problem does the project address? Detecting micro-cracks, scratches, contamination, and structural defects in industrial parts where defective samples are scarce.

• How is it handled currently? 1) Manual human visual inspection, which suffers from fatigue, high labor costs, and subjective inconsistency; and 2) Supervised deep learning (YOLO/CNNs).

• Supervised models require thousands of labeled defect examples and fail silently on novel defect forms.

## Difficulties & Gaps Aimed to Reduce

• Defect Scarcity Bottleneck: Factory scrap rates are <0.1%; collecting balanced defect datasets is economically impossible.

• Unbounded Flaw Variations: Defects take infinite unknown visual shapes.

• Accuracy-Latency Trade-off Gap: PatchCore provides ~99% AUROC but has high query latency; EfficientAD provides millisecond speed but requires heavy GPU training.

• Lack of Explainable Localization: Operators need pixel heatmaps, not just black-box PASS/FAIL labels.

# Motivation



## Why Choose This Project?

• High Industry Impact: Visual inspection is a top-tier commercial application of computer vision in aerospace, automotive, and pharmaceuticals.

• Deep Algorithmic Value: Offers practical implementation experience with intermediate PyTorch layer hooks, dimensionality reduction, and greedy coreset algorithms.

• Safety-Critical Need: A single undetected defect in aircraft fasteners or medical vials causes severe failures, costly recalls, and life-safety hazards.

## Who Benefits From This Project?

• Quality Assurance Engineers: Automated 24/7 inspection with objective, mathematical consistency.

• Factory Floor Operators: Real-time visual heatmaps pinpointing exact flaw locations instantly.

• Manufacturing System Architects: Objective engineering data on whether to deploy memory banks or student-teacher models based on hardware constraints.

# Objectives



## Core Technical Objectives (1 - 3)

• 1. Design and Implement PatchCore from scratch in PyTorch, extracting multi-scale patch features from pre-trained convolutional backbones (layer2 and layer3) without wrapper libraries.

• 2. Optimize the patch memory bank using Johnson-Lindenstrauss random projection and greedy k-Center coreset subsampling, achieving a 99% reduction in stored vectors.

• 3. Validate and Compare PatchCore across all 15 MVTec AD categories and benchmark against EfficientAD across representative classes.

## Applied & Deployment Objectives (4 - 5)

• 4. Calculate and Analyze standard industrial metrics: Image-level AUROC, Pixel-level AUROC, per-sample inference latency (ms), and GPU/CPU memory consumption.

• 5. Develop and Display an interactive Streamlit inspection application featuring real-time JET defect heatmaps, PASS/DEFECT classification, and automated SQLite audit logging.

# Scope



## Features Included in Project Scope

- Complete from-scratch PyTorch implementation of PatchCore (feature extraction, neighborhood pooling, greedy k-center).
- MVTec AD dataset ingestion (15 categories: 5 textures, 10 objects) with pixel-level ground truth mask alignment.
- Benchmarking EfficientAD student-teacher models on 5 representative categories (bottle, cable, capsule, metal_nut, transistor).
- Robustness evaluation under synthetic lighting shifts (+/- 20% brightness) and sample-size reduction (10%, 25%, 50%, 100%).
- Interactive Streamlit GUI with SQLite exception logging.

## What is Outside the Project Scope

- Physical camera hardware mounting and conveyor belt mechanical trigger integration.
- Industrial fieldbus communication protocols (e.g., Modbus, Profinet, EtherCAT).
- Full training of EfficientAD across all 15 categories due to academic GPU compute limits.
- 3D point cloud and volumetric sensor processing (MVTec 3D-AD).

# Methodology



## Data Pipeline & Feature Extraction

• Step 1: Dataset Pipeline — Standardize MVTec AD images to 224x224, apply ImageNet normalization, and pair test anomalies with ground-truth binary masks.

• Step 2: Multi-Scale Patch Extraction — Attach PyTorch forward hooks to layer2 and layer3 of ResNet18/WideResNet50. Apply 3x3 stride-1 average pooling to aggregate local neighborhoods into 384-D patch tokens.

• Step 3: Coreset Memory Bank Construction — Project features to 128-D via Johnson-Lindenstrauss. Run Min-Max Greedy k-Center to compress 163,856 patches to 1,638 vectors (1.0% coreset).

## Distillation, Benchmarking & App

• Step 4: EfficientAD Knowledge Distillation — Train student network and autoencoder to mimic a fixed pre-trained teacher network exclusively on normal parts.

• Step 5: Comparative Evaluation & Robustness Testing — Benchmark detection accuracy, millisecond latency, and few-shot degradation.

• Step 6: Dashboard & Audit Trail Deployment — Build Streamlit UI for visual inspection with automated SQLite logging.

# Working Principle



## Step-by-Step Workflow Mechanics

• 1. User Input: Operator uploads an inspection image or chooses an online sample from the MVTec AD test stream.

• 2. Patch Extraction: Input image passes through convolutional backbone; multi-scale neighborhood patch descriptors are extracted.

• 3. Distance Calculation: Each query patch computes Euclidean distance to its nearest neighbor in the stored normal memory bank: s_i = min || p_i - m ||.

• 4. Heatmap Generation: Patch distances are bilinearly upsampled to 224x224 and smoothed using a 2D Gaussian filter (sigma = 4).

## Output Produced & Audit Logging

• 5. Visual Output: Anomaly map normalized and converted to a JET colormap overlay: Display = 0.6 * Image + 0.4 * Heatmap.

• 6. Verdict Decision: Maximum patch distance represents image anomaly score. Score >= Threshold -> DEFECT; Score < Threshold -> PASS.

• 7. Telemetry & Database Record: Automatically commits timestamp, part category, anomaly score, threshold, verdict, and latency into inspections.db.

# Applications



## High-Reliability Industrial Domains

• Aerospace & Defense: Surface inspection of turbine blades, fastener integrity, composite panels, and machined engine casings.

• Pharmaceutical & Healthcare: Detection of cracked glass vials, particulate contamination in liquid medicine, and damaged blister packs.

• Semiconductor & PCB Manufacturing: Automated detection of short circuits, missing solder joints, bridge defects, and scratched wafers.

## Automotive & Fast-Moving Consumer Goods

• Automotive Assembly: Quality inspection of engine piston rings, brake discs, welded chassis seams, and exterior paint finish.

• Packaging & Bottling Lines: Monitoring bottle seal completeness, capping alignment, label positioning, and structural wall cracks.

• Textile & Material Production: Weave irregularities, surface abrasions, and color deviations in carpet, leather, and tile production.

# Advantages



## Algorithmic & Operational Benefits

• 1. Zero Defect Training Data Needed: Eliminates defect collection bottlenecks by learning entirely from normal, defect-free parts.

• 2. Unseen Flaw Detection: Accurately flags novel, unexpected defect types that supervised classifiers miss.

• 3. 99% Memory Reduction via Coreset Optimization: Greedy k-center reduces memory bank from 163,856 to 1,638 patches without performance loss.

## Speed, Explainability & Traceability

• 4. Zero-Epoch Instant Deployment: PatchCore requires no backpropagation training, allowing immediate deployment.

• 5. Pixel-Level Visual Explainability: Generates intuitive JET defect heatmaps rather than black-box binary decisions.

• 6. Regulatory Compliance: Automated SQLite logging fulfills ISO 9001 and aerospace QA traceability standards.

# Limitations



## Computational & Architectural Constraints

- 1. Higher Latency on CPU Hardware: PatchCore nearest-neighbor search takes ~2.2 - 2.5 seconds on CPU, requiring GPU/INT8 quantization for fast conveyor speeds.
- 2. Training Overhead for EfficientAD: Requires ~70,000 backpropagation iterations per category, requiring dedicated GPU training compute.
- 3. Memory Footprint Scales with Categories: Storing separate patch banks for 15 classes requires substantial RAM/VRAM.

## Visual & Environmental Sensitivity

- 4. Sensitivity to Extreme Misalignment: Large rotational misalignments or viewpoint changes can produce false alarms.
- 5. 2D Surface Constraint: Cannot detect internal sub-surface voids or volumetric structural flaws that require X-ray/ultrasound sensors.

# Conclusion



## Summary of What Was Developed

- Developed an end-to-end unsupervised visual defect inspection framework for industrial quality control.
- Implemented PatchCore from scratch in PyTorch with multi-scale layer hooks and greedy k-Center subsampling.
- Built a functional Streamlit inspection interface with real-time JET heatmaps and automated SQLite audit logging.
- Established a clear empirical baseline comparing memory-bank methods against student-teacher networks.

## Objectives Met & Main Learning Outcomes

- Objectives Achieved: All 5 primary objectives achieved—scratch implementation, 99% coreset reduction, empirical validation, metric evaluation, and GUI deployment.
- Empirical Validation: Verified clean separation on MVTec 'bottle' (Defect score: 3.28 vs Normal score: 1.30 against threshold 2.50).
- Key Learning: Mastered the practical engineering trade-offs between zero-shot memory-bank accuracy and millisecond distillation latency.

# Future Scope



## Edge Deployment & Architectural Improvements

• 1. Edge Hardware Quantization: Apply INT8 post-training quantization and TensorRT compilation to drop CPU latency from ~2500ms to <50ms for Raspberry Pi / Jetson deployment.

• 2. Unified Multi-Class Model: Develop a single unified memory bank / distillation network that inspects all 15 MVTec classes simultaneously without switching checkpoints.

• 3. 3D Vision & Point Cloud Integration: Expand framework to MVTec 3D-AD by fusing RGB images with depth point clouds to detect 3D surface dents.

## Factory Automation & PLC Connectivity

• 4. Industrial PLC Bus Integration: Connect SQLite audit logger to industrial communication protocols (Modbus / OPC-UA) to trigger pneumatic rejection arms on factory sorters.

• 5. Active Learning Feedback Loop: Enable quality operators to confirm false alarms, dynamically updating the coreset memory bank.

# References



**1. PatchCore (Main Benchmark Paper):**
Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). Towards Total Recall in Industrial Anomaly Detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14298–14308.

**2. EfficientAD (Comparison Architecture):**
Batzner, K., Heckler, L., & König, R. (2024). EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 127–137.

**3. MVTec AD (Benchmark Dataset):**
Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2019). MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9584–9592.

**4. PaDiM (Baseline Comparison):**
Defard, T., Setkov, A., Loesch, A., & Audigier, R. (2021). PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization. In International Conference on Pattern Recognition (ICPR Workshops), Springer LNCS 12664, pp. 475–489.

**5. VisA (Benchmark Dataset & Self-Supervision):**
Zou, Y., Jeong, J., Pemula, L., Zhang, D., & Dhome, M. (2022). SPot-the-Difference: Self-Supervised Pre-training for Anomaly Detection and Segmentation (VisA). In European Conference on Computer Vision (ECCV), Springer LNCS 13690, pp. 392–408.